Blogging System API

Version

Version: 1.0

Project Overview

The Blogging System API is a RESTful backend application built using **ASP.NET Core Web API** and **C#**. The system provides a secure blogging platform where administrators can publish and manage blog posts while registered users can interact with published content through threaded discussions.

The application emphasizes secure authentication, clean architecture, proper software engineering practices, and reliable data persistence using PostgreSQL. It is designed as a backend-only service that can later be consumed by any frontend application, including web, desktop, or mobile clients.

Project Objectives

The objectives of this project are to:

- Develop a secure RESTful API using ASP.NET Core.
- Implement robust authentication and authorization using JWT.
- Support role-based access control for administrators and users.
- Store application data in a PostgreSQL database using Entity Framework Core.
- Secure user passwords using BCrypt hashing.
- Implement email-based One-Time Password (OTP) verification for account registration and password recovery.
- Provide a scalable threaded commenting system.
- Support Markdown-based blog content.
- Instrument the application using OpenTelemetry for observability.
- Produce comprehensive technical documentation before implementation.

Project Scope

The system will include the following major modules:

Authentication

- User registration
- Email OTP verification

- User login
- JWT Access Token generation
- Refresh Token generation
- Forgot password
- Password reset using OTP
- Change password

User Management

- User registration
- User authentication
- User profile management
- Role management
- Email verification

Blog Management

Administrators will be able to:

- Create blog posts
- Update blog posts
- Delete blog posts
- Save posts as drafts
- Publish drafts
- Lock or unlock comments on individual posts

Users will be able to:

- View published posts

Comment Management

Users can:

- Comment on blog posts
- Reply to comments
- Reply to replies with unlimited nesting

Administrators can:

- Reply to comments on their own posts
- Lock discussions on posts they own

User Roles

Administrator

An Administrator is responsible for managing blog content.

Permissions include:

- Create posts
- Edit own posts
- Delete own posts
- Save drafts
- Publish drafts
- Reply to comments on owned posts
- Lock and unlock comments

User

A User is responsible for consuming blog content.

Permissions include:

- Register an account
- Verify account using OTP
- Log in
- Read published posts
- Comment on posts
- Reply to comments
- Reply to replies
- Change password
- Reset forgotten password

Technology Stack

Component	Technology
Language	C#
Framework	ASP.NET Core Web API
Runtime	.NET SDK
Database	PostgreSQL

Component	Technology
ORM	Entity Framework Core
Authentication	JWT
Password Hashing	BCrypt
Email Service	SMTP
Logging	OpenTelemetry
API Documentation	Swagger / OpenAPI
API Testing	Postman
Version Control	Git
Content Format	Markdown

Project Characteristics

The application is designed with the following principles:

- RESTful API architecture
- Backend-only implementation
- Role-based authorization
- Secure authentication
- Layered architecture
- Database normalization
- Clean and maintainable code
- Comprehensive documentation
- Test-driven feature validation

Out of Scope (Version 1)

The following features are intentionally excluded from Version 1:

- Image uploads
- User-generated blog posts
- Social reactions (likes/dislikes)
- Categories and tags
- Full-text content search
- File attachments
- Notifications
- Real-time messaging

- Public user profiles
- Third-party authentication (Google, GitHub, Facebook)
- Docker deployment
- Cloud hosting

These features may be considered in future versions.

Success Criteria

The project will be considered complete when:

- All functional requirements have been implemented.
- Authentication is fully operational.
- Role-based authorization is enforced.
- All data is persisted in PostgreSQL.
- Passwords are securely hashed.
- Email OTP verification functions correctly.
- CRUD operations for blog posts are complete.
- Threaded commenting is operational.
- OpenTelemetry logging is configured and exporting to the console.
- All API endpoints are documented and tested using Postman.
- The application builds and runs successfully without errors.